# Simulador de Ecosistema

Informe técnico. Aplicación de escritorio Java Swing con arquitectura MVC + DAO, interfaz diseñada en NetBeans y persistencia de partidas en archivos de texto.




INTEGRANTES · ANALISTA EN SISTEMAS

VC **Valentina Caramanico**
Integrante

GM **Giovanna Marandino**
Integrante

PC **Paloma Corbalán**
Integrante

Docente: **Giuliana Dealbera Etchechoury**

Segunda instancia evaluativa · Entrega: **4 de noviembre de 2026**

ÍNDICE

# Contenido del informe

01 **Introducción** — Del simulador de consola a la aplicación de escritorio — 3
02 **Objetivos** — General y específicos — 3
03 **Alcance del sistema** — Qué permite hacer la aplicación — 4
04 **Reglas funcionales del ecosistema** — Entidades, clima y turnos — 4
05 **Configuración de la simulación** — Parámetros, rangos y validación — 6
06 **Arquitectura del sistema** — Modelo, Vista, Controlador y DAO — 6
07 **Descripción de las clases principales** — Responsabilidad de cada clase — 7
08 **Interfaz gráfica y navegación** — Pantallas y recorrido — 8
09 **Persistencia de datos** — Guardar y recuperar partidas — 9
10 **Reportes y estadísticas** — Evolución e informe final — 10
11 **Validaciones y manejo de errores** — Control de datos y excepciones — 11
12 **Pruebas funcionales** — Recorridos verificados — 11
13 **Diseño UX/UI y prototipo Figma** — Persona, decisiones y prototipo — 12
14 **Dificultades y soluciones** — Desafíos y cómo se resolvieron — 13
15 **Uso de herramientas de inteligencia artificial** — Apoyo, proceso y evidencias — 14
16 **Conclusiones** — Cierre del trabajo — 15
17 **Referencias** — Enlaces y documentación — 15

SECCIÓN 01

# Introducción

01

El presente informe describe el desarrollo de la segunda instancia evaluativa del proyecto **Simulador de Ecosistema**, tomando como punto de partida la aplicación realizada durante IE1.

El objetivo principal de esta etapa fue transformar el simulador original, cuya interacción se realizaba mediante consola, en una aplicación de escritorio con interfaz gráfica desarrollada utilizando Java Swing y el entorno Apache NetBeans.

El sistema representa un ecosistema compuesto por tres tipos principales de entidades: plantas, conejos y lobos. Estas interactúan entre sí y con las condiciones ambientales durante una cantidad determinada de turnos.

La nueva versión incorpora pantallas para configurar y ejecutar simulaciones, visualizar la evolución de las poblaciones, intervenir sobre el ecosistema y guardar o recuperar partidas mediante archivos de texto.

Además, se reorganizó el proyecto utilizando una arquitectura basada en las capas Modelo, Vista, Controlador y DAO, buscando mejorar la separación de responsabilidades, la comprensión del código y su mantenimiento.

| 3 | 4 | 4 | .txt |
|---|---|---|---|
| ESPECIES SIMULADAS | CLIMAS | CAPAS: MVC + DAO | PERSISTENCIA |

**TECNOLOGÍAS**

Java, Swing, NetBeans, Git y archivos de texto.

**REPOSITORIO**

GitHub — Interfaz gráfica IE2

SECCIÓN 02

# Objetivos

02

## 2.1 Objetivo general

Transformar el simulador de ecosistema desarrollado en IE1 en una aplicación de escritorio utilizando Java Swing, conservando las reglas funcionales originales e incorporando una interfaz gráfica, persistencia de datos y una organización del código por capas.

## 2.2 Objetivos específicos

- Implementar una interfaz gráfica mediante Java Swing.
- Permitir la configuración inicial de las poblaciones y del clima.
- Visualizar el estado del ecosistema durante la simulación.

- Permitir avanzar la simulación turno a turno.
- Incorporar intervenciones del usuario durante la ejecución.
- Implementar el guardado y la recuperación de partidas.
- Mostrar estadísticas sobre la evolución de las poblaciones.
- Generar un reporte final con información relevante del ecosistema.
- Organizar el código utilizando una arquitectura MVC y DAO.

SECCIÓN 03

# Alcance del sistema

03

La aplicación permite iniciar una nueva simulación, configurar sus parámetros iniciales, observar la evolución del ecosistema y realizar intervenciones durante su ejecución.

También permite guardar el estado actual en un archivo de texto y recuperar posteriormente la simulación para continuar desde el turno almacenado.

### FUNCIONALIDADES PRINCIPALES

| | | |
|---|---|---|
| ⚙️ **Configuración inicial** | ⏭️ **Ejecución por turnos** | 📊 **Visualización de poblaciones** |
| 🌦️ **Visualización del clima** | 📝 **Registro de eventos** | 👷 **Intervención del usuario** |
| 💾 **Guardado y carga de simulaciones** | 📈 **Consulta de estadísticas** | 📋 **Generación de reportes finales** |

SECCIÓN 04

# Reglas funcionales del ecosistema

04

El ecosistema está compuesto por plantas, conejos y lobos.

### 🌿 Plantas

Pueden crecer y reproducirse de acuerdo con las condiciones definidas en el modelo. También existen plantas venenosas, que producen efectos particulares cuando son consumidas.

### 🐇 Conejos

Buscan alimento y modifican su energía según las condiciones de alimentación. Su supervivencia depende de la disponibilidad de recursos y de las condiciones ambientales.

### 🐺 Lobos

Realizan intentos de caza sobre los conejos. El resultado de estas acciones repercute en sus niveles de energía y en los indicadores de actividad del ecosistema.

Las entidades también envejecen, consumen energía y pueden morir cuando dejan de cumplir sus condiciones de supervivencia.

## 4.1 Clima

El clima influye sobre el comportamiento y las condiciones del ecosistema. Se contemplan cuatro alternativas:

☀️ **Soleado** · 🌧️ **Lluvioso** · 🏜️ **Sequía** · ❄️ **Invierno**

## 4.2 Procesamiento de cada turno

1. **Procesar acciones** — Las acciones correspondientes a cada entidad
2. **Actualizar estado** — Del ecosistema
3. **Verificar finalización** — Condiciones de fin




**Figura 1.** Registro de un evento de planta venenosa (“Conejo-3 comió una planta venenosa. Perdió 20 puntos de energía”) y diálogo de finalización de la simulación.

SECCIÓN 05

# Configuración de la simulación

05

La aplicación permite configurar los parámetros iniciales mediante controles gráficos.

| PARÁMETRO | MÍNIMO | MÁXIMO |
|---|---|---|
| **Plantas** | 5 | 30 |
| **Conejos** | 2 | 15 |
| **Lobos** | 1 | 5 |
| **Turnos** | 10 | 50 |

El clima inicial se selecciona mediante una lista desplegable.

**DOBLE CONTROL**

Los valores se controlan tanto desde los componentes gráficos como desde el controlador, evitando que se inicie una simulación con parámetros fuera de los rangos establecidos.




**Figura 2.** Pantalla de configuración: selectores numéricos, lista de climas y botones Iniciar Simulación / Volver.

SECCIÓN 06

# Arquitectura del sistema

06

El proyecto se organiza en cuatro paquetes principales: **modelo**, **vista**, **controlador** y **dao**. Esta estructura permite distribuir las responsabilidades y evitar concentrar toda la lógica del sistema en una única clase.



### 6.1 Modelo

modelo

Contiene las clases que representan las entidades del ecosistema y su comportamiento. El modelo constituye el núcleo lógico de la aplicación.

- Administrar las poblaciones.
- Representar los atributos de las entidades.
- Ejecutar las reglas de comportamiento.
- Procesar los turnos.
- Controlar nacimientos y muertes.
- Registrar eventos y estadísticas.
- Determinar las condiciones de finalización.

### 6.2 Vista

vista

Contiene las ventanas y componentes gráficos desarrollados con Java Swing. Se ocupa de presentar información y recibir las acciones del usuario.

- `FrmPrincipal`: pantalla inicial y acceso a las operaciones principales.
- `FrmConfiguracion`: define los parámetros de una nueva simulación.
- `FrmSimulacion`: presenta el estado del ecosistema y permite avanzar los turnos.
- `FrmEstadisticas`: evolución de las poblaciones mediante una tabla y un gráfico.

### 6.3 Controlador

controlador

La clase `ControladorSimulacion` coordina las operaciones entre la interfaz y el modelo. La interfaz utiliza sus métodos en lugar de concentrar las reglas del ecosistema.

- Iniciar simulaciones y validar parámetros.
- Procesar los turnos.
- Consultar el estado de las poblaciones.
- Cambiar el clima y agregar entidades.
- Determinar si una simulación finalizó.
- Coordinar el guardado y la recuperación.
- Proporcionar información para reportes y estadísticas.

### 6.4 DAO

dao

La clase `SimulacionDAO` escribe y lee la información necesaria para conservar el estado de una simulación, sin depender de los componentes gráficos.

- Turno actual y cantidad de turnos configurada.
- Clima.
- Poblaciones y atributos de las entidades.
- Información de las plantas venenosas.
- Estadísticas acumuladas y eventos registrados.
- Historial de poblaciones.
- Información necesaria para reconstruir la partida.

SECCIÓN 07

# Descripción de las clases principales

07

#### 7.1 ECOSISTEMA

Es la clase central del modelo. Administra las poblaciones y coordina la ejecución de los turnos. También mantiene información sobre el estado de la simulación, los eventos ocurridos y la evolución de las poblaciones.

#### 7.2 ENTIDAD

Representa la abstracción general de los elementos que forman parte del ecosistema. A partir de esta estructura se desarrollan las distintas clases concretas.

#### 7.3 PLANTA, CONEJO Y LOBO

Representan los distintos tipos de entidades y sus comportamientos particulares.

#### 7.4 CONTROLADORSIMULACION

Coordina las operaciones que solicita la interfaz y mantiene la comunicación con el modelo y la persistencia.

#### 7.5 SIMULACIONDAO

Implementa las operaciones de guardado y recuperación de simulaciones mediante archivos de texto.

**PROGRAMACIÓN ORIENTADA A OBJETOS**

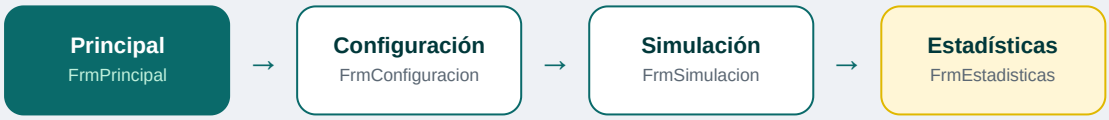
Las clases conservan los principios trabajados en IE1: **herencia**, **abstracción**, **encapsulamiento** y **polimorfismo**.

SECCIÓN 08

# Interfaz gráfica y navegación

08

La interfaz se desarrolló utilizando Java Swing y el diseñador visual de NetBeans. Se buscó ofrecer una navegación sencilla, con pantallas diferenciadas para cada etapa del funcionamiento.




- La **pantalla principal** permite acceder a una nueva simulación o cargar una previamente guardada.
- La **pantalla de configuración** permite establecer los parámetros iniciales.
- Durante la **simulación** se muestran los contadores de plantas, conejos y lobos, el turno actual, el clima y el registro de eventos.
- También se incorporaron **imágenes representativas** de las entidades para facilitar su identificación visual.
- La **pantalla de estadísticas** permite consultar los datos históricos mediante una tabla y una representación gráfica.

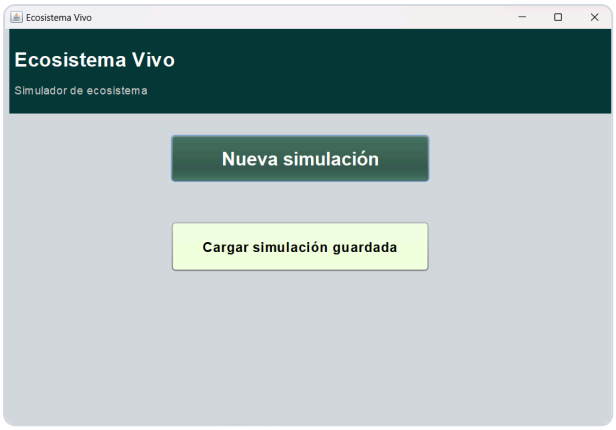



**Figura 3.** Pantalla principal: nueva simulación o carga de una partida guardada.




**Figura 4.** Simulación en curso: clima, contadores por especie, turno y registro de eventos.

SECCIÓN 09

# Persistencia de datos

09

Una de las funcionalidades principales incorporadas en esta instancia es la persistencia de las simulaciones. El sistema permite guardar una partida en un archivo de texto y posteriormente recuperarla.

Para realizar esta operación, el DAO serializa la información necesaria para reconstruir las entidades y el estado del ecosistema. Al cargar una partida, el controlador recupera la información y permite continuar la ejecución desde el turno almacenado.

### QUÉ CONTIENE EL ARCHIVO

- **Turno** — Turno actual y turnos configurados
- **Clima** — Condición ambiental
- **Entidades** — Poblaciones y atributos
- **Venenosas** — Plantas venenosas
- **Estadísticas** — Acumuladas
- **Eventos** — Registrados
- **Historial** — De poblaciones
- **Estado** — Datos para reconstruir la partida

### GUARDAR

La vista avisa al controlador → el controlador coordina con el DAO → el DAO escribe el archivo de texto con el estado de la simulación.

### CARGAR

El DAO lee el archivo → el controlador recupera la información → la simulación continúa desde el turno almacenado.

### VERIFICACIÓN

Se realizaron pruebas de guardado y recuperación, verificando la continuidad de la simulación.

SECCIÓN 10

# Reportes y estadísticas

10

El sistema incorpora mecanismos para analizar la evolución del ecosistema.

### PANTALLA DE ESTADÍSTICAS

- Historial de poblaciones por turno.
- Cantidad máxima de plantas, conejos y lobos.
- Cantidad mínima de cada población.
- Turnos en los que se registraron los máximos y mínimos.
- Gráfico de evolución de las poblaciones.

### REPORTE FINAL

Reúne información sobre el desarrollo de la simulación, incluyendo los indicadores solicitados en la consigna:

- Causa de finalización.
- Turno de mayor actividad.
- Entidad más longeva de cada tipo.
- Lobo con mayor cantidad de cazas exitosas.
- Cantidad de nacimientos y muertes por tipo.

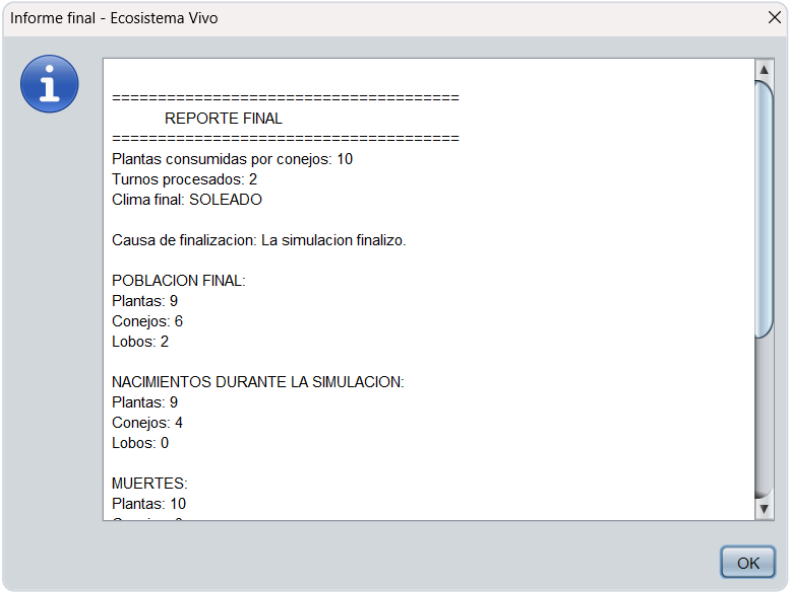



**Figura 5.** Reporte final mostrado en una ventana al concluir la simulación.

SECCIÓN 11

# Validaciones y manejo de errores

11

### PARÁMETROS DE CONFIGURACIÓN

El sistema incorpora validaciones para controlar los parámetros de configuración y evitar valores fuera de los límites permitidos.

### SIMULACIÓN INICIADA

También contempla verificaciones antes de ejecutar operaciones que requieren una simulación iniciada.

### PERSISTENCIA

En las operaciones de persistencia se utilizan mecanismos de manejo de excepciones para informar cuando ocurre un problema durante la lectura o escritura de archivos.

SECCIÓN 12

# Pruebas funcionales

12

Durante el desarrollo se realizaron pruebas sobre los principales recorridos de la aplicación.

| PRUEBA | RESULTADO |
|---|---|
| Clean and Build del proyecto | CORRECTO |
| Inicio de una nueva simulación | CORRECTO |
| Configuración de parámetros | CORRECTO |
| Avance de turnos | CORRECTO |
| Actualización de poblaciones | CORRECTO |
| Registro de eventos | CORRECTO |
| Visualización de estadísticas | CORRECTO |
| Visualización del gráfico | CORRECTO |
| Guardado de una simulación | CORRECTO |
| Recuperación de una partida guardada | CORRECTO |
| Continuación desde el turno recuperado | CORRECTO |

ALCANCE DE LAS PRUEBAS

Estas pruebas corresponden a los recorridos verificados durante el desarrollo. Los casos de archivos dañados, permisos insuficientes y todos los indicadores del reporte final deberán documentarse según las pruebas efectivamente realizadas antes de la entrega.

SECCIÓN 13

# Diseño UX/UI y prototipo Figma

13

El diseño de la interfaz se acompaña con un prototipo navegable desarrollado en Figma. El prototipo permite representar las pantallas principales y el recorrido esperado de la persona usuaria.

ENLACE AL PROTOTIPO

Prototipos Ecosistema IE2 — Figma

## 13.1 Quién usa la aplicación



**Lucía**

17 años · estudiante de nivel secundario

PERSONA Y ESCENARIO

**Quién:** una estudiante curiosa por la biología y la tecnología, con manejo básico de computadora.

**Escenario:** en una clase de ciencias quiere observar cómo cambia un ecosistema cuando varía el clima o cuando se agregan lobos, y comparar los resultados de cada partida.

**Necesita:** configurar rápido, ver cuántos ejemplares hay de cada especie, entender qué pasó en cada turno y retomar una partida guardada.

## 13.2 Decisiones de diseño

### Un botón principal por pantalla

1

JERARQUÍA VISUAL

La acción principal es grande y usa el verde azulado de la marca; las secundarias son claras y más pequeñas.

### Un color por especie

2

RECONOCER ANTES QUE RECORDAR

Plantas en verde, conejos en amarillo y lobos en rojo permiten leer el estado de un vistazo.

### Misma estructura en todas las pantallas

3

CONSISTENCIA Y ESTÁNDARES

Banda superior oscura con el título, misma tipografía y paleta en todas las vistas.

### Controles que evitan el error

4

PREVENCIÓN DE ERRORES

Selectores numéricos y lista de climas en lugar de texto libre, con validación en el controlador.

**El sistema siempre informa qué está pasando** 5

VISIBILIDAD DEL ESTADO DEL SISTEMA

Turno actual, clima, contadores y registro de eventos; diálogos que confirman cuándo termina la partida.

## 13.3 Del prototipo a la aplicación

Al comienzo del proyecto se crearon prototipos que resultaron **muy complejos de llevar a cabo y materializar en NetBeans**, por las limitaciones del diseñador visual y de los layouts, y porque la consigna exige construir la interfaz sin crear componentes por código. Por eso el diseño se simplificó, conservando lo esencial: banda superior oscura, botón principal destacado, contadores de colores por especie y registro de eventos.

SECCIÓN 14

# Dificultades y soluciones 14

Durante el desarrollo se presentaron distintos desafíos técnicos.

DESAFÍO

**14.1 Adaptación de consola a interfaz gráfica.** Fue necesario trasladar la interacción del usuario a ventanas Swing, manteniendo las reglas del modelo original.

SOLUCIÓN

Creación de formularios diferenciados para la configuración, ejecución y consulta de estadísticas.

DESAFÍO

**14.2 Integración entre capas.** La incorporación de la interfaz requirió organizar la comunicación entre el modelo y las ventanas.

SOLUCIÓN

Se trabajó sobre `ControladorSimulacion` para centralizar las operaciones y exponer métodos de consulta a las vistas.

DESAFÍO

**14.3 Persistencia y recuperación.** El guardado inicial no contemplaba toda la información necesaria para reconstruir una partida.

SOLUCIÓN

Se amplió el DAO para conservar los datos de las entidades, sus atributos y la información histórica necesaria para continuar la simulación.

DESAFÍO

**14.4 Edición mediante el diseñador visual.** Hubo que cuidar el código generado automáticamente por NetBeans.

SOLUCIÓN

Los cambios se realizaron principalmente en los métodos editables, conservando los bloques administrados por el diseñador.

DESAFÍO

**14.5 Reporte y datos de un reporte guardado.** Fue de los puntos con más conflictos: generar el reporte y recuperar los datos de una partida guardada.

SOLUCIÓN

Se corrigió la validación de la cabecera del archivo y se amplió la información que se guarda, verificando la continuidad de la simulación.

DESAFÍO

**14.6 Del modelo de IE1 al de IE2.** Adaptar las clases a las nuevas estructuras de capas llevó tiempo y varios ajustes.

SOLUCIÓN

Se separaron responsabilidades entre modelo, vista, controlador y DAO, conservando las reglas originales.

DESAFÍO

**14.7 Errores en los primeros turnos.** Lo más difícil fue identificar errores de funcionamiento durante los primeros turnos después de cambiar la estructura física del proyecto.

SOLUCIÓN

Se probó turno a turno después de cada cambio, analizando cada falla con ayuda de ChatGPT y los cheatsheets de la docente.

DESAFÍO

**14.8 Prototipos complejos.** Los primeros prototipos eran difíciles de materializar en NetBeans por sus limitaciones.

SOLUCIÓN

Se simplificó el diseño manteniendo la jerarquía, la consistencia y los colores por especie (ver 13.3).

SECCIÓN 15

# Uso de herramientas de inteligencia artificial

15

Durante el desarrollo se utilizaron herramientas de inteligencia artificial, principalmente **ChatGPT**, como apoyo para:

- Analizar errores de compilación.
- Comprender problemas de integración entre clases.
- Revisar métodos del controlador y del DAO.
- Mejorar la organización del código.
- Preparar documentación técnica.
- Organizar las pruebas y la explicación del proyecto.

CÓMO LO USAMOS

1. **Entender el problema** — ChatGPT explica la lógica de lo que había que resolver
2. **Pasarlo a código** — Explica la lógica a implementar a nivel de código
3. **Contrastar** — Cheatsheets de la docente y documentación
4. **Probar y corregir** — Revisar los primeros turnos tras cada cambio

Las sugerencias generadas fueron revisadas, adaptadas e incorporadas al proyecto después de comprobar su funcionamiento. El uso de estas herramientas no reemplazó la revisión del código ni las pruebas realizadas por el equipo.

**CARPETA PROMPTS**

Las partes más importantes de la comunicación con ChatGPT se encuentran en la carpeta `prompts` del repositorio de GitHub, con la documentación individual de uso de IA.

**CARPETA VIDEOS**

Los videos explicativos se encuentran en la carpeta `VIDEOS` del repositorio de GitHub.

SECCIÓN 16

# Conclusiones

16

La segunda instancia evaluativa permitió transformar el simulador original en una aplicación de escritorio con interfaz gráfica, conservando las principales reglas funcionales del ecosistema.

La incorporación de Java Swing facilitó la interacción con el sistema y permitió presentar información de forma más clara mediante ventanas, indicadores, tablas y gráficos.

La implementación de persistencia representó un avance importante, ya que permite guardar el estado de una simulación y recuperarlo posteriormente para continuar su ejecución.

Asimismo, la organización en paquetes separados permitió trabajar sobre las responsabilidades del modelo, la vista, el controlador y el acceso a datos.

El desarrollo permitió aplicar conceptos de programación orientada a objetos, arquitectura de software, persistencia, diseño de interfaces y pruebas funcionales.

Como resultado, se obtuvo una aplicación funcional que constituye una evolución de la versión de consola desarrollada en IE1.

SECCIÓN 17

# Referencias

17

**01** Dealbera Etchechoury, Giuliana. *Interfaz Gráfica — Segunda instancia evaluativa: Proyecto Simulador de Ecosistema versión Swing*. Consigna de la materia, 2026.

**02** Documentación oficial de Java. https://docs.oracle.com/en/java/

**03** Documentación de Java Swing. https://docs.oracle.com/javase/tutorial/uiswing/

**04** Apache NetBeans. https://netbeans.apache.org/

**05** Figma. *Prototipos Ecosistema IE2*. https://www.figma.com/design/BFfixiiBMYoSJ0QhFdhzeP/Prototipos-Ecosistema-IE2?…

**06** Repositorio del proyecto (código, carpetas `prompts` y `VIDEOS`). https://github.com/Giomarandino/Interfaz-grafica---IE2

FIN DEL INFORME